

HelpDesk-AWS

AWS Cloud Deployment Project

Project Report

Prepared by: Madin Maksud Shindi

Project Type: Cloud Engineer / AWS Portfolio Project

AWS Region: ap-south-1 (Mumbai)

Technologies:

**AWS • Linux • Python • Flask • MySQL • Amazon EC2 • Amazon RDS • Amazon S3 •
Application Load Balancer • Auto Scaling • VPC • CloudWatch • IAM • Git • GitHub**

Table of Contents

1. Introduction
 2. Project Objectives
 3. Project Overview
 4. Technologies and AWS Services Used
 5. Application Overview
 6. AWS Architecture
 7. VPC and Network Configuration
 8. Subnet Configuration
 9. Internet Gateway and Routing
 10. Security Groups and Network Security
 11. IAM Configuration
 12. EC2 Deployment
 13. Flask Application Deployment
 14. Application Load Balancer
 15. Target Group and Health Checks
 16. Amazon RDS MySQL Database
 17. Database Configuration and Testing
 18. Amazon S3
 19. CloudWatch Monitoring
 20. Auto Scaling and Launch Template
 21. Testing and Validation
 22. Problems Encountered and Troubleshooting
 23. Security Considerations
 24. Cost Management and Resource Cleanup
 25. Skills Demonstrated
 26. Project Outcome
 27. Conclusion
-

1. Introduction

The HelpDesk AWS Cloud Deployment Project was developed as a practical cloud engineering portfolio project to demonstrate the deployment and operation of a web-based HelpDesk application using Amazon Web Services.

The project combines cloud infrastructure, Linux administration, application deployment, database management, networking, load balancing, monitoring and basic scalability.

The application was developed using Python and Flask and provides a simple HelpDesk system where users can submit support tickets. Ticket information is stored in a MySQL database hosted using Amazon RDS.

The application infrastructure was deployed inside a custom Amazon VPC. Amazon EC2 instances were used to host the Flask application, while an Application Load Balancer was configured to distribute incoming HTTP traffic across the application instances.

Amazon S3 was also incorporated into the project for cloud object storage, while CloudWatch was used for monitoring and operational visibility.

The project was designed as a practical hands-on exercise rather than a production enterprise system. The main objective was to understand how the individual AWS services work together to provide a complete cloud-hosted application environment.

2. Project Objectives

The main objectives of the HelpDesk project were:

- Deploy a Python Flask application on Amazon EC2.
- Build a custom VPC for the application infrastructure.
- Configure public subnets across multiple Availability Zones.
- Configure Internet Gateway and route tables.
- Implement Security Groups to control network traffic.
- Deploy a MySQL database using Amazon RDS.
- Connect the Flask application to the RDS database.
- Store cloud-based application data using Amazon S3.
- Configure an Application Load Balancer.
- Configure an EC2 Target Group and health checks.
- Implement an Auto Scaling Group and Launch Template.
- Use CloudWatch for monitoring.
- Practice Linux server administration.

- Test application availability through the load balancer.
 - Understand troubleshooting of unhealthy targets and application failures.
 - Clean up AWS resources after completing the project to minimise unnecessary costs.
-

3. Project Overview

The HelpDesk application is a simple web-based support ticket system.

Users can submit tickets containing information such as:

- Name
- Email address
- Subject
- Description
- Ticket status
- Creation timestamp

The Flask backend receives requests from the web application and stores ticket information in a MySQL database.

The application architecture was divided into several cloud components.

At the networking layer, a custom VPC was created with a CIDR range of:

10.0.0.0/24

Two public subnets were configured across different Availability Zones:

- ap-south-1a
- ap-south-1b

Two EC2 instances were deployed to host the Flask application.

An Application Load Balancer was placed in front of the EC2 instances. The load balancer forwarded HTTP requests to port **5000**, where the Flask application was running.

Amazon RDS was used to provide the MySQL database backend.

4. Technologies and AWS Services Used

Development Technologies

Python

Python was used as the primary programming language for the backend application.

Flask

Flask was used as the web framework for creating the HelpDesk application and API endpoints.

MySQL

MySQL was used as the relational database system for storing HelpDesk tickets.

Linux

Amazon Linux was used as the operating system for the EC2 servers.

Git and GitHub

Git was used for source-code version control and GitHub was used to store the project source code.

AWS Services

Amazon VPC

Used to create the isolated networking environment for the HelpDesk infrastructure.

Amazon EC2

Used to host the Flask application.

Amazon RDS

Used to provide the managed MySQL database.

Amazon S3

Used for cloud object storage.

Application Load Balancer

Used to distribute HTTP requests between EC2 instances.

Target Groups

Used to register EC2 instances and perform application health checks.

Auto Scaling

Used to demonstrate automated EC2 instance management and scalable infrastructure.

CloudWatch

Used for monitoring AWS resources and application infrastructure.

IAM

Used to control access to AWS resources.

5. Application Overview

The HelpDesk application provides a basic support-ticket workflow.

A user submits a support request through the application.

The Flask backend receives the request and stores it inside the MySQL database.

The database contains a `tickets` table with the following fields:

Field	Type	Purpose
id	INT	Unique ticket identifier
name	VARCHAR(100)	User name
email	VARCHAR(150)	User email
subject	VARCHAR(200)	Ticket subject
description	TEXT	Ticket description
status	VARCHAR(30)	Current ticket status

created_at TIMESTAMP Ticket creation time

The default ticket status was configured as:

Open

A test ticket was successfully inserted into the database.

6. AWS Architecture

The overall architecture consisted of the following flow:

User → Internet → Application Load Balancer → Target Group → EC2 Instances → Flask Application → Amazon RDS MySQL

Additional AWS services such as S3, CloudWatch and IAM supported the application environment.

The infrastructure was deployed in the Mumbai AWS Region (**ap-south-1**).

The custom VPC used the CIDR block:

10.0.0.0/24

Two subnets were created:

HelpDesk-Public-1

CIDR: **10.0.0.0/26**

Availability Zone: **ap-south-1a**

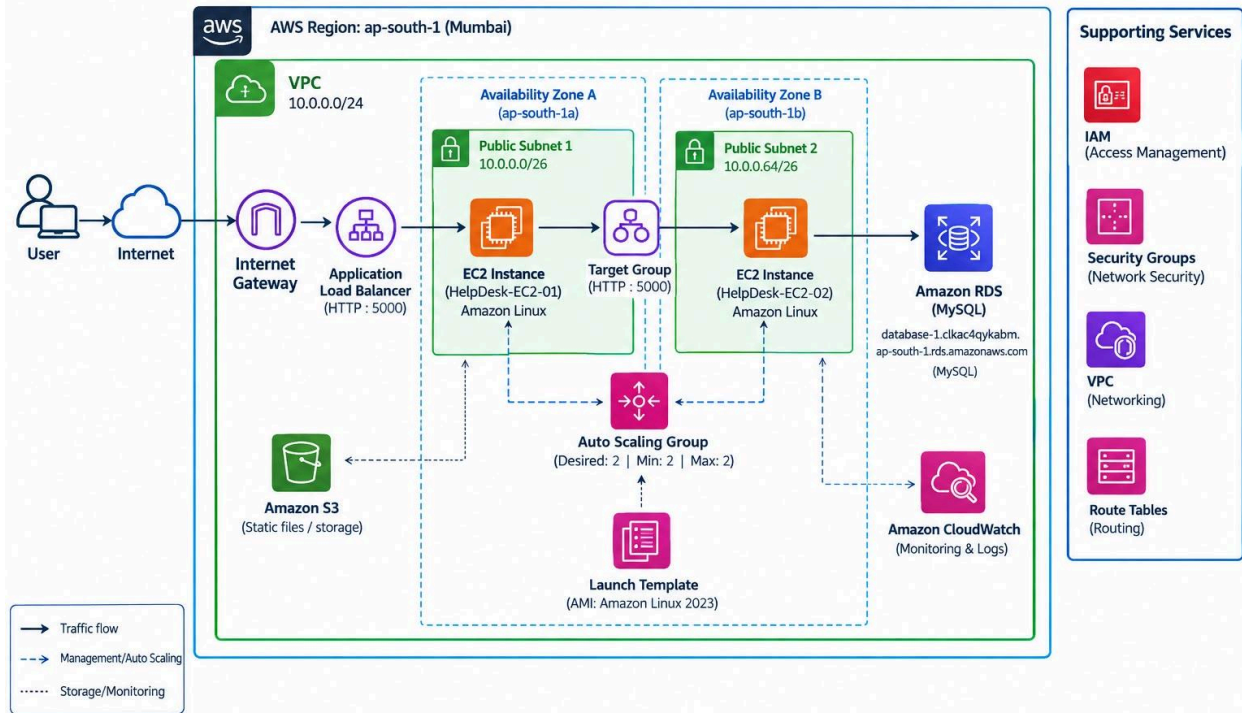
HelpDesk-Public-2

CIDR: **10.0.0.64/26**

Availability Zone: **ap-south-1b**

The two Availability Zones provided a basic demonstration of multi-AZ application deployment.

HelpDesk – AWS Cloud Architecture



7. VPC and Network Configuration

A dedicated VPC named **HelpDesk-VPC** was created for the project.

The VPC used the following configuration:

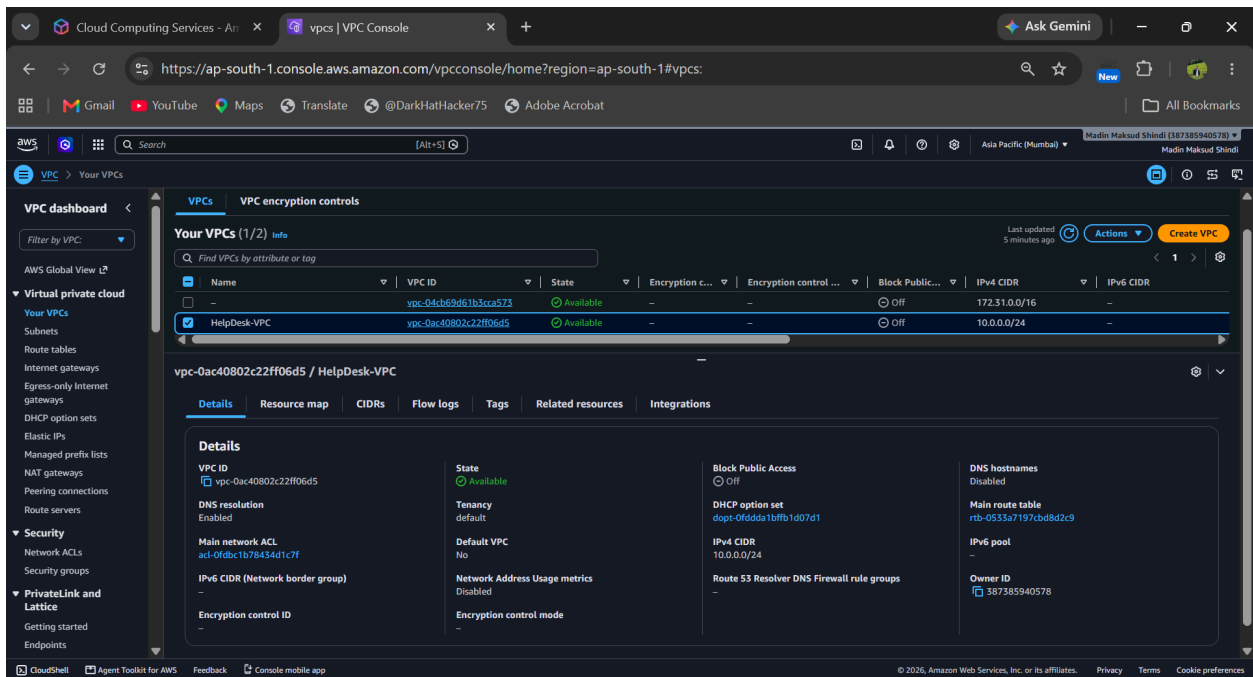
VPC Name: HelpDesk-VPC

CIDR: 10.0.0.0/24

Region: ap-south-1

The purpose of using a dedicated VPC was to provide control over the networking environment rather than placing the application directly into the default AWS networking environment.

The VPC formed the foundation for the EC2, load balancer and database connectivity.



8. Subnet Configuration

Two subnets were created in separate Availability Zones.

Subnet 1

Name: HelpDesk-Public-1

CIDR: 10.0.0.0/26

Availability Zone: ap-south-1a

Subnet 2

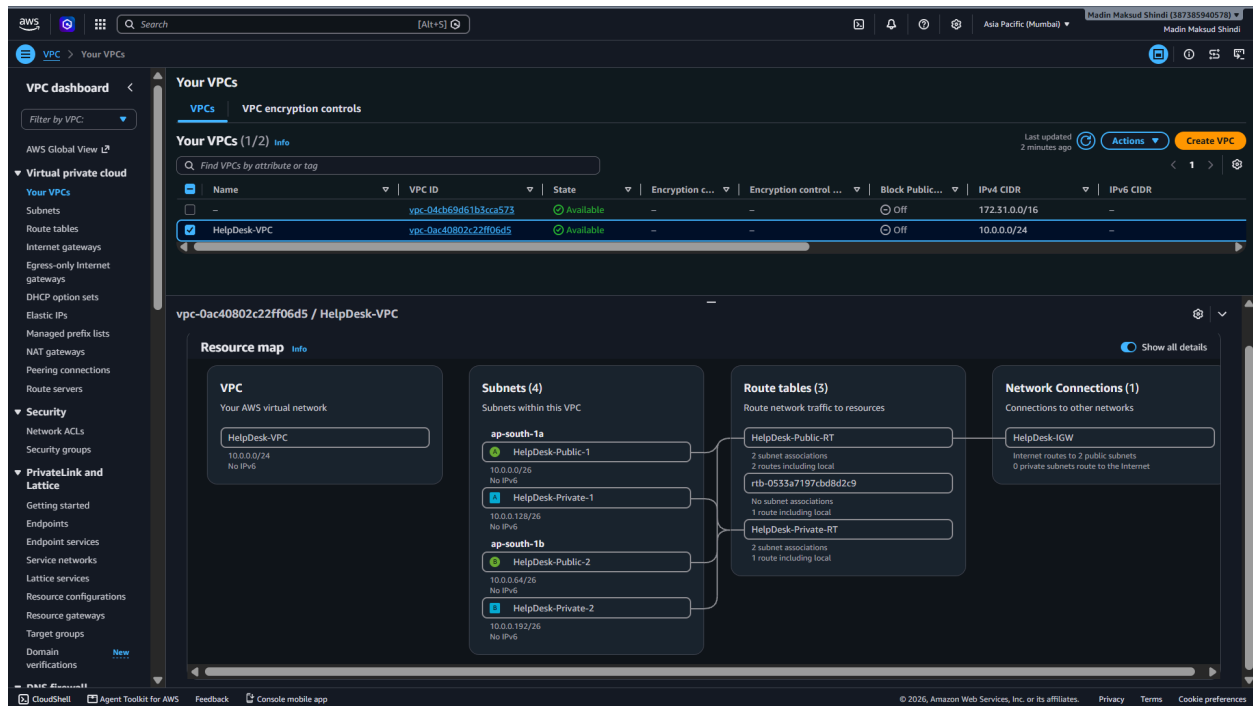
Name: HelpDesk-Public-2

CIDR: 10.0.0.64/26

Availability Zone: ap-south-1b

Using two Availability Zones provided the opportunity to distribute application instances across separate AWS infrastructure locations.

This configuration was later used by the Application Load Balancer and Auto Scaling Group.



9. Internet Gateway and Routing

An Internet Gateway was configured for the HelpDesk VPC.

The Internet Gateway provided a path between the VPC and the public internet.

A route table was configured with an internet route:

Destination: 0.0.0.0/0

Target: Internet Gateway

The route table was associated with the public subnets.

This allowed resources in the public subnets with appropriate public IP addressing and security rules to communicate with the internet.

10. Security Groups and Network Security

Security Groups were used as virtual firewalls for the application infrastructure.

A Security Group named:

HelpDesk-EC2-SG

was associated with the EC2 instances.

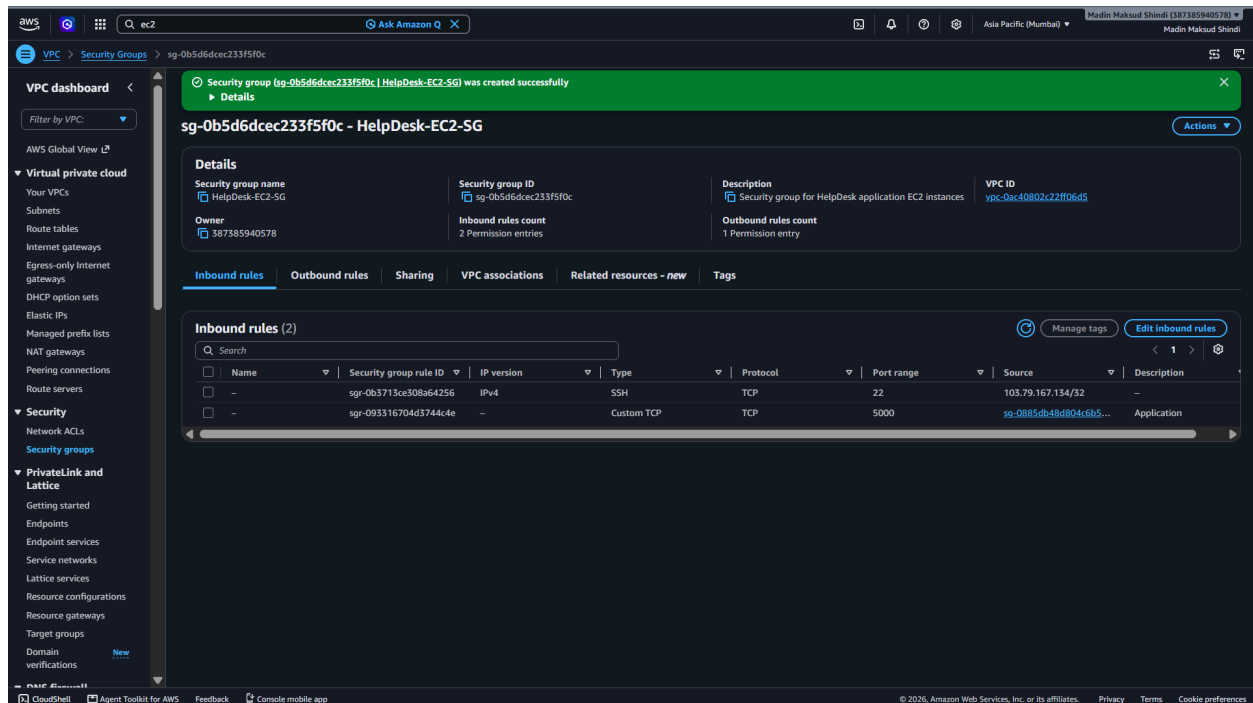
The application operated on:

TCP port 5000

The load balancer communicated with the application instances using HTTP on port 5000.

The security configuration was designed to allow the required application traffic while avoiding unnecessary open ports.

The project demonstrated the importance of controlling traffic using AWS Security Groups rather than exposing every service to the internet.



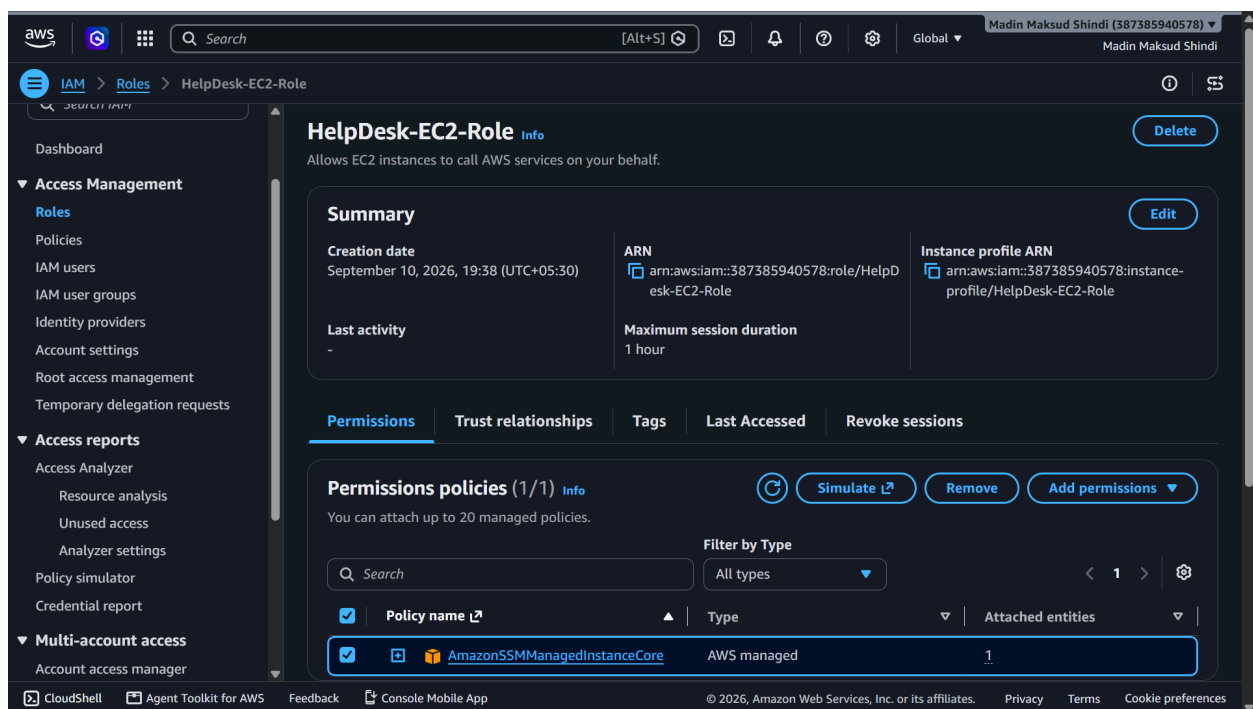
11. IAM Configuration

AWS Identity and Access Management was used to control permissions for AWS resources.

An EC2 IAM role was configured for the application infrastructure where required.

IAM roles are preferable to storing long-term AWS access keys directly on EC2 instances because temporary credentials can be provided to applications through the EC2 instance metadata service.

The project also demonstrated the importance of applying permissions only when they are required by the application.



12. EC2 Deployment

Amazon EC2 was used to host the Flask application.

Two application instances were initially deployed:

HelpDesk-EC2-01

HelpDesk-EC2-02

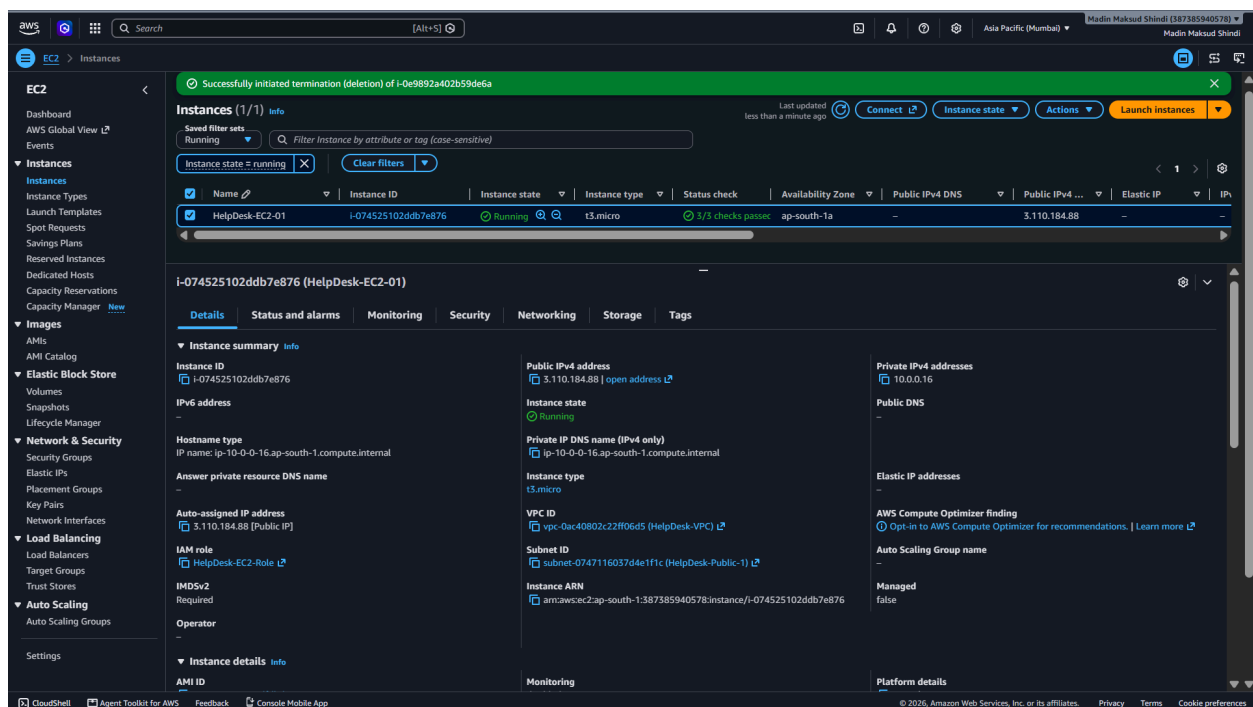
The instances were distributed between:

- ap-south-1a
- ap-south-1b

Amazon Linux was used as the operating system.

The servers were accessed using SSH and administered through the Linux command line.

Basic Linux operations were performed to install dependencies, configure the application and test connectivity.



13. Flask Application Deployment

The Flask application was deployed directly onto the EC2 instances.

Python dependencies were installed on the server.

The MySQL Python connector was also installed and verified.

The following command was used to verify the connector:

```
python3 -c "import mysql.connector; print('MySQL connector OK')"
```

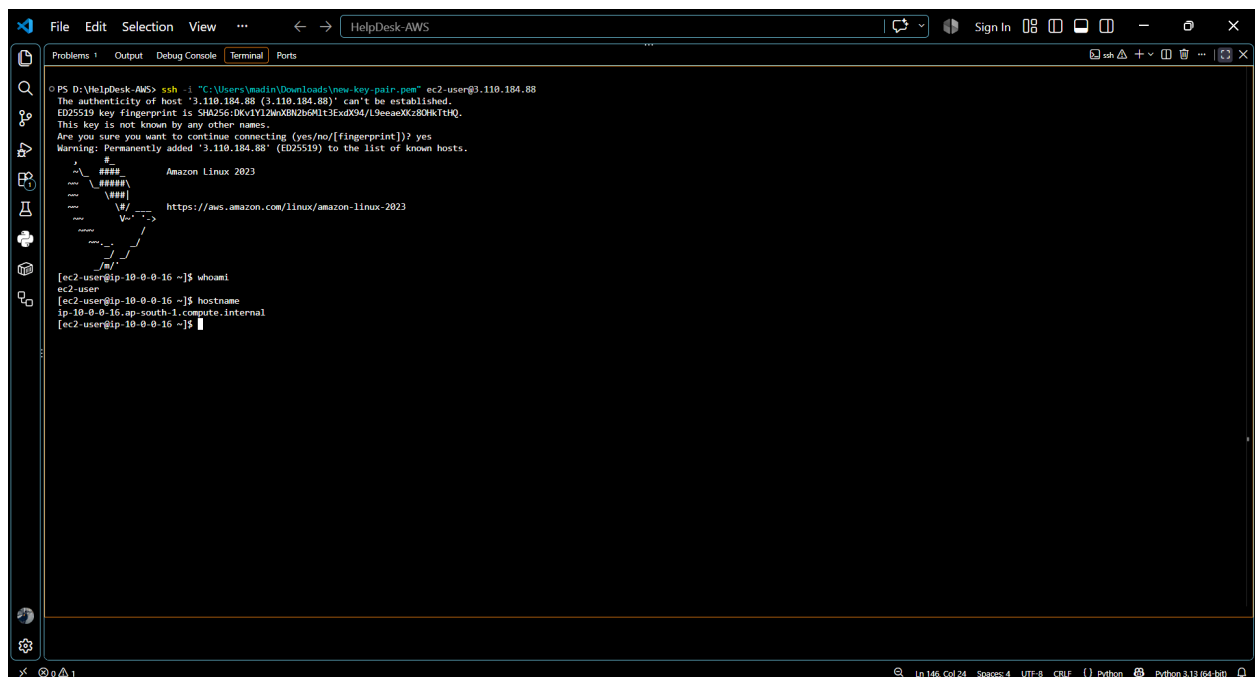
The successful output confirmed that the Python environment could import the MySQL connector.

The Flask application was configured to listen on:

0.0.0.0:5000

This was important because binding only to **127.0.0.1** would make the application accessible only from the local server.

Binding to **0.0.0.0** allowed traffic from the Application Load Balancer to reach the application.



The screenshot shows a terminal window titled 'HelpDesk-AWS'. The user is running an SSH command from a Windows PowerShell prompt: `ssh -i "C:\Users\maddin\Downloads\new-key-pair.pem" ec2-user@3.110.184.88`. The terminal output shows the SSH connection process, including the warning about the host's authenticity and the user's confirmation to continue. The terminal then displays the Amazon Linux 2023 logo and the URL <https://aws.amazon.com/linux/amazon-linux-2023>. Finally, the user runs three commands: `whoami` (output: `ec2-user`), `hostname` (output: `ip-10-0-0-16.ap-south-1.compute.internal`), and `ip` (output: `ec2-user@ip-10-0-0-16 ~$`).

14. Application Load Balancer

An Application Load Balancer named:

HelpDesk-ALB

was created.

The load balancer provided a single entry point for users while distributing requests to the EC2 application instances.

The ALB was configured for HTTP traffic.

The application traffic was forwarded to the target group:

HelpDesk-TG

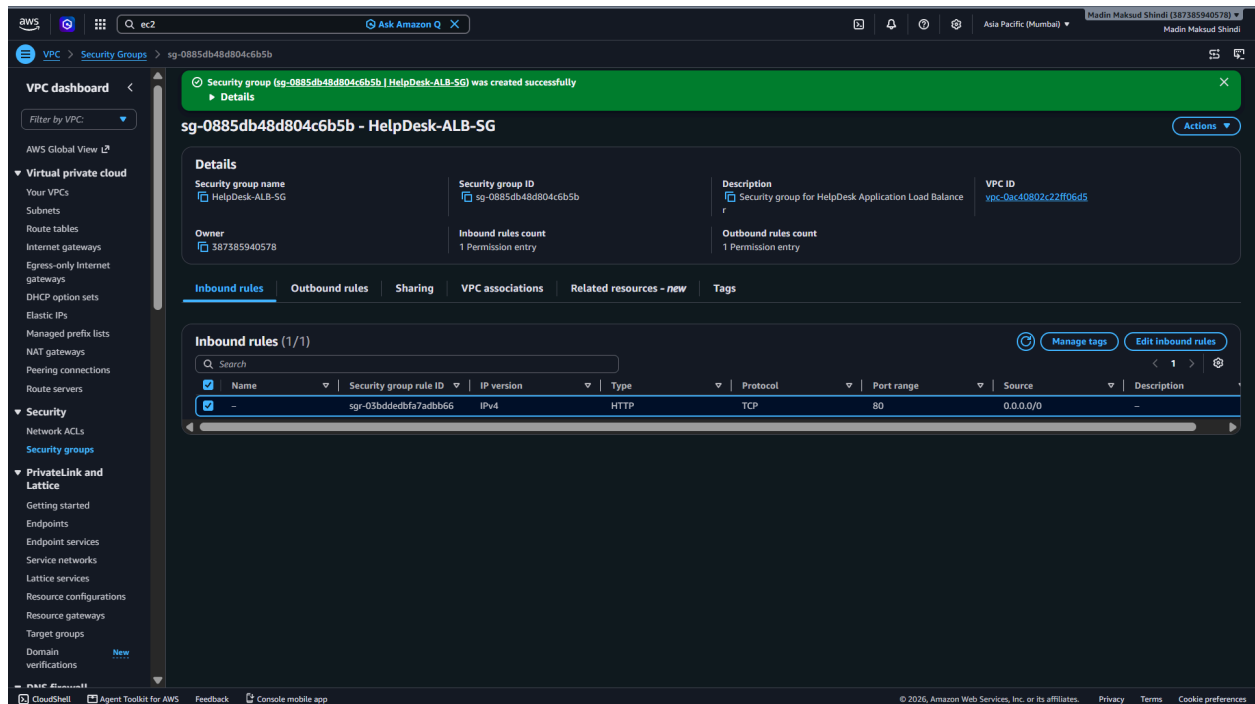
The target group used:

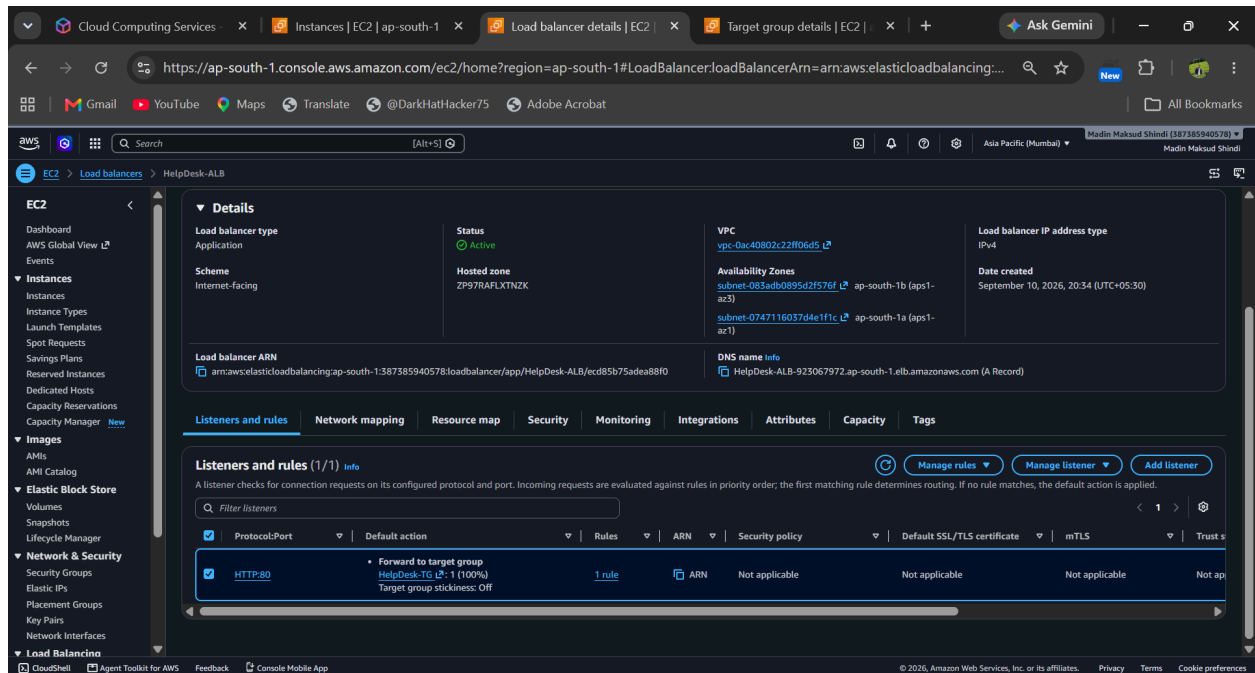
Protocol: HTTP

Port: 5000

This allowed the ALB to forward requests to the Flask application running on each EC2 instance.

The ALB architecture also demonstrated how application traffic can be separated from individual server addresses.





15. Target Group and Health Checks

The target group was named:

HelpDesk-TG

The target type was:

Instance

The protocol and port were:

HTTP:5000

The target group used application health checks to determine whether an EC2 instance was capable of receiving traffic.

The Flask application included a `/health` endpoint.

The endpoint was tested locally on the EC2 server using:

```
curl http://localhost:5000/health
```

A successful HTTP response demonstrated that the Flask application was running correctly on the manually configured application server.

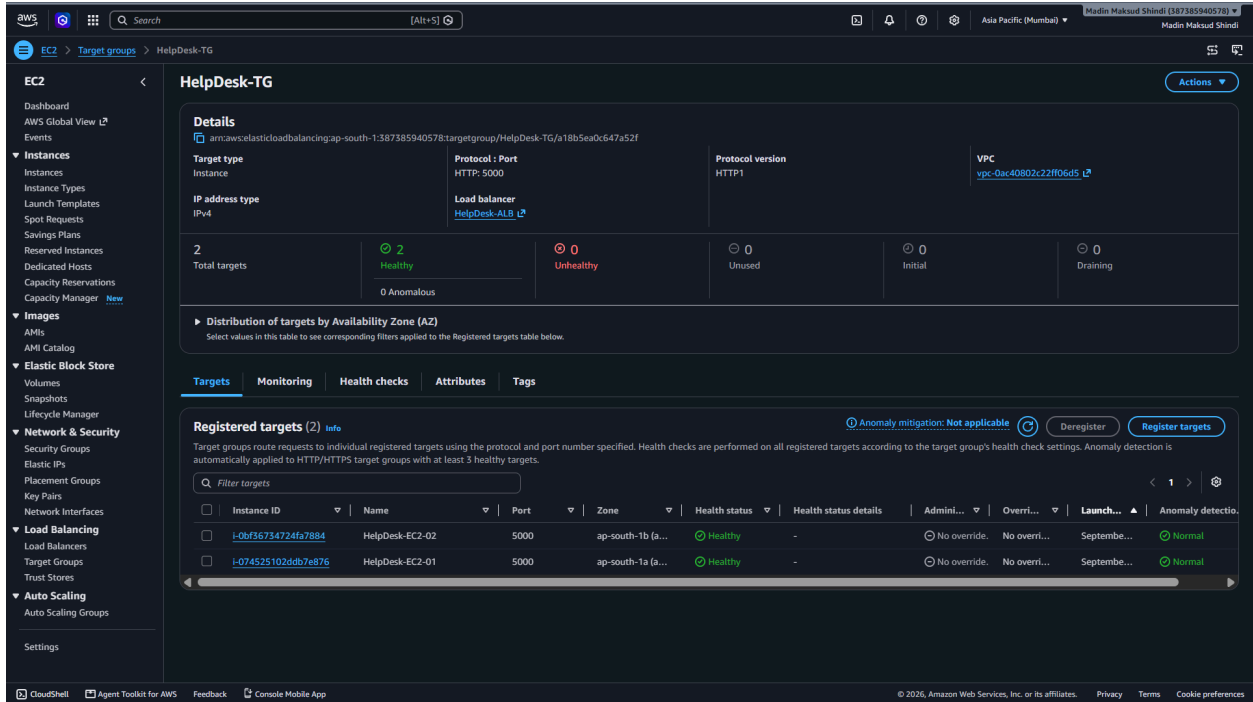
The ALB continuously used health checks to determine which targets were healthy.

This was an important demonstration of the difference between an EC2 instance being technically running and an application actually being available.

The screenshot displays the AWS Management Console interface for the 'ap-south-1' region. The 'Target groups' page is active, showing a table with one target group: 'HelpDesk-TG'. The table columns include Name, ARN, Port, Protocol, Target type, Load balancer, and VPC ID. The 'HelpDesk-TG' target group is associated with the 'arn:aws:elasticloadbalancing:ap-south-1:387385940578:targetgroup/HelpDesk-TG/a18b5ea0c647a52f' ARN, port 5000, HTTP protocol, Instance target type, HelpDesk-ALB load balancer, and vpc-0ac40802c22ff06d5 VPC ID.

Below the table, the 'Target group: HelpDesk-TG' details are shown. The 'Details' section includes the target type (Instance), protocol (HTTP), port (5000), protocol version (HTTP1), and VPC (vpc-0ac40802c22ff06d5). The 'IP address type' is set to IPv4. The 'Health' status is 'Healthy' with 1 total target and 0 anomalous targets. The 'Unhealthy' status is 'Unhealthy' with 0 total targets. The 'Unused' status is 'Unused' with 0 total targets. The 'Initial' status is 'Initial' with 0 total targets. The 'Draining' status is 'Draining' with 0 total targets.

The 'Distribution of targets by Availability Zone (AZ)' section is also visible, showing a table with columns for Availability Zone, Number of targets, and Percentage of targets.



16. Amazon RDS MySQL Database

Amazon RDS was used to provide the application's managed relational database.

A MySQL RDS database was created in the Mumbai region.

The database instance was configured with a small development-oriented instance size suitable for hands-on practice.

The database was not intended for production workloads.

The application connected to the RDS endpoint instead of running the database directly on the EC2 server.

The RDS endpoint was:

database-1.clkac4qykabm.ap-south-1.rds.amazonaws.com

Using Amazon RDS demonstrated the advantages of separating application compute from database infrastructure.

AWS handled much of the underlying database infrastructure, while the application interacted with the database through its endpoint.

17. Database Configuration and Testing

The MySQL client available on the EC2 server was verified.

The installed client reported MariaDB-compatible MySQL client software.

A database named:

helpdesk

was created.

The following table was created:

tickets

The table contained seven fields:

- id
- name
- email
- subject
- description
- status
- created_at

The table structure was verified using:

DESCRIBE tickets;

A test ticket was then inserted.

Example test data:

Name: Test User

Email: test@example.com

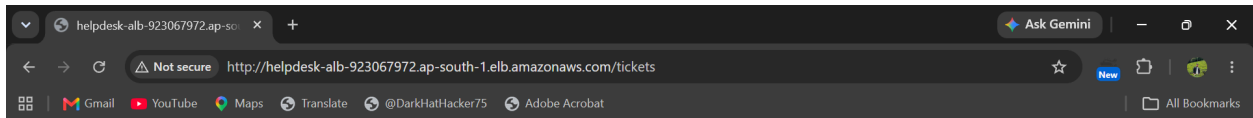
Subject: Login Issue

Description: Unable to log in to the HelpDesk system.

The database successfully returned the inserted record when queried using:

```
SELECT * FROM tickets;
```

This confirmed that the RDS database was reachable and that ticket records could be created successfully.



HelpDesk Tickets

ID: 1

Name: Test User

Email: test@example.com

Subject: Login Issue

Description: Unable to log in to the HelpDesk system.

Status: Open

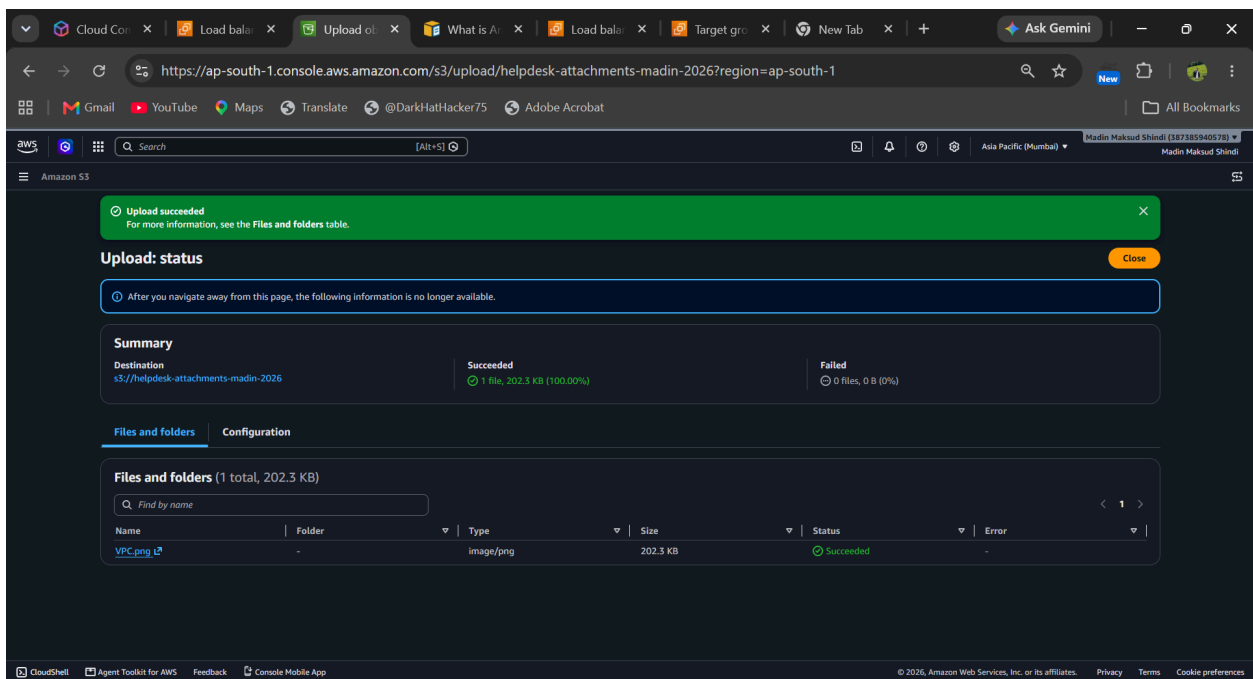
18. Amazon S3

Amazon S3 was included in the HelpDesk architecture to demonstrate cloud-based object storage.

An S3 bucket was created for the project and used as part of the application's cloud storage environment.

S3 provides highly durable object storage and can be used for assets such as documents, images, application files and other objects.

For this portfolio project, the S3 component demonstrated how application infrastructure can use a separate managed storage service instead of relying entirely on local EC2 storage.



19. CloudWatch Monitoring

Amazon CloudWatch was used for monitoring the AWS environment.

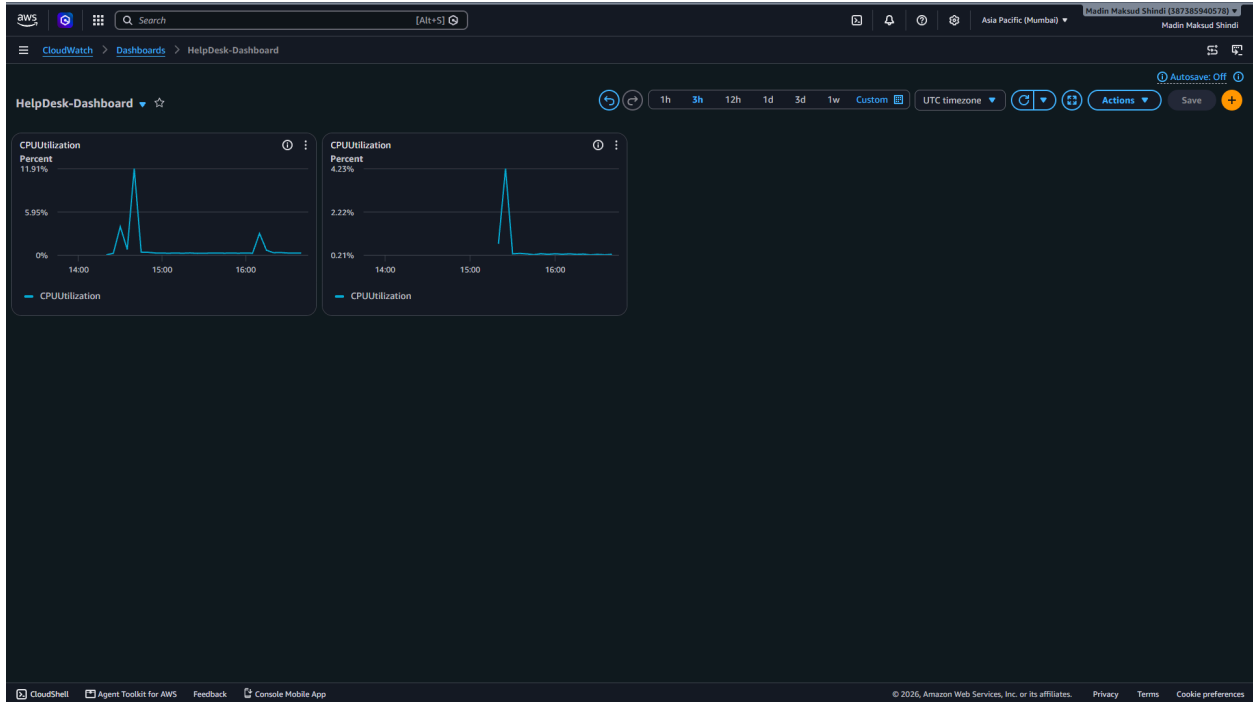
CloudWatch provides visibility into AWS resource metrics and can be used to monitor infrastructure health.

The project used CloudWatch to observe EC2 and application infrastructure.

Monitoring is important because cloud engineers need to understand not only whether infrastructure exists, but whether it is operating correctly.

Metrics such as CPU utilisation and instance health can help identify resource problems.

CloudWatch also works alongside other AWS services such as Auto Scaling and can be used to create alarms and automated responses.



20. Auto Scaling and Launch Template

An Auto Scaling Group named:

HelpDesk-ASG

was created.

A Launch Template named:

HelpDesk-Launch-Template

was configured for the Auto Scaling Group.

The Launch Template specified the basic EC2 configuration, including:

- Amazon Linux 2023
- t3.micro instance type
- SSH key pair
- HelpDesk Security Group
- EC2 configuration

The Auto Scaling Group was configured with:

Desired capacity: 2

Minimum capacity: 2

Maximum capacity: 2

The Auto Scaling Group was also associated with the existing Application Load Balancer target group.

This demonstrated how EC2 infrastructure can be managed through an Auto Scaling Group rather than manually creating every instance.

The configuration used two Availability Zones:

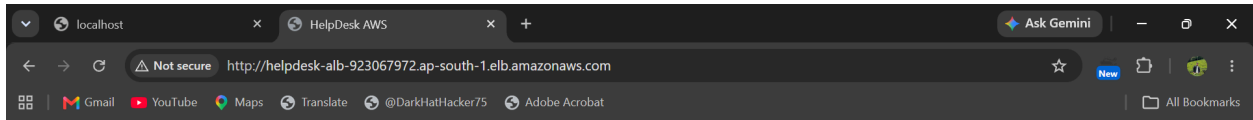
- ap-south-1a
 - ap-south-1b
-

21. Testing and Validation

Several tests were performed throughout the project.

Application Test

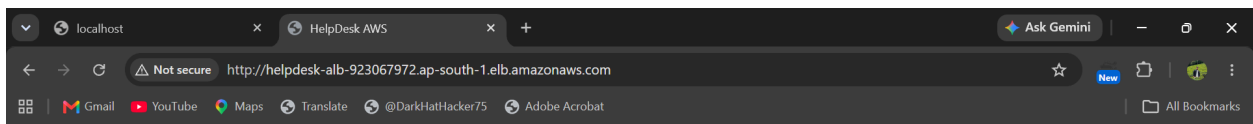
The Flask application was started on the EC2 server and verified using the application URL.



HelpDesk AWS

EC2-01

Help Desk application is running successfully.



HelpDesk AWS

EC2-02

Help Desk application is running successfully.

Health Check Test

The health endpoint was tested using:

```
curl http://localhost:5000/health
```

The endpoint returned a successful response while the Flask application was running.

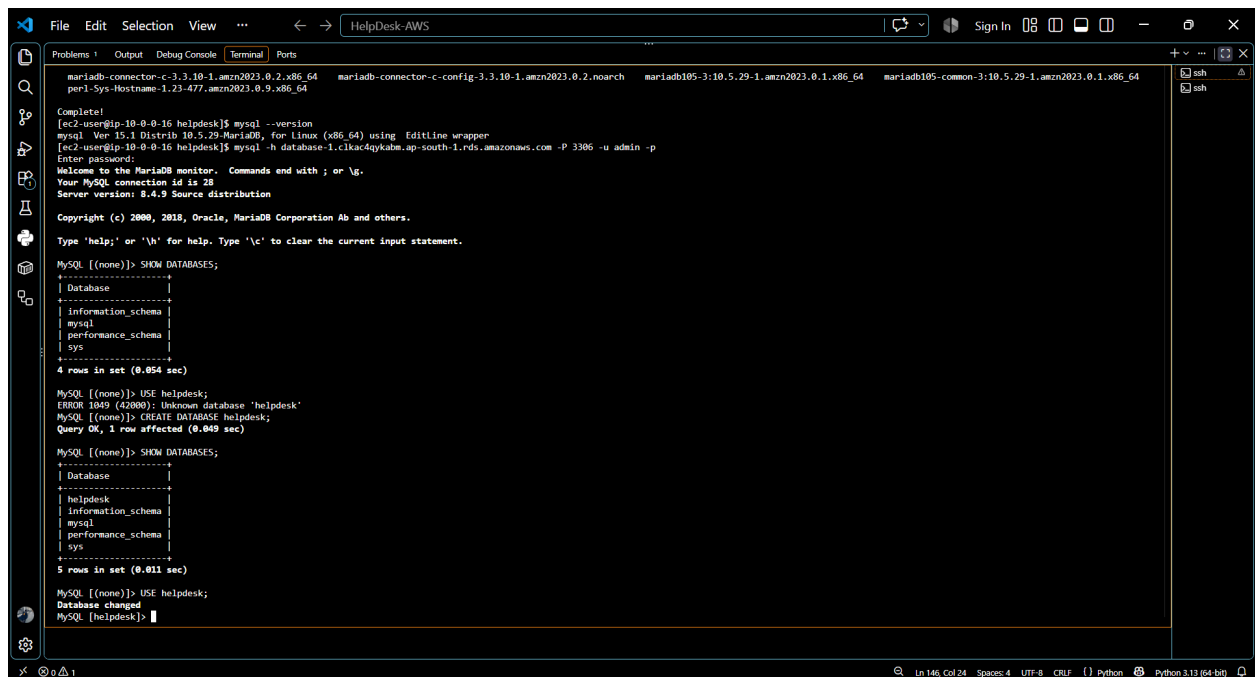
The ALB also generated health-check requests against the target instances.

Database Test

A test database and table were created.

A test support ticket was inserted and retrieved successfully.

This verified database connectivity and basic CRUD functionality.



```
File Edit Selection View ... HelpDesk-AWS
Problems Output Debug Console Terminal Ports
mariadb-connector-c-3.3.10-1.amzn2023.0.2.x86_64 mariadb-connector-c-config-3.3.10-1.amzn2023.0.2.noarch mariadb105-3:10.5.29-1.amzn2023.0.1.x86_64 mariadb105-common-3:10.5.29-1.amzn2023.0.1.x86_64
perl-Sys-Hostname-1.23-477.amzn2023.0.9.x86_64

Complete!
[ec2-user@ip-10-0-0-16 helpdesk]$ mysql --version
mysql Ver 15.1 Distrib 10.5.29-MariaDB, for Linux (x86_64) using Editline wrapper
[ec2-user@ip-10-0-0-16 helpdesk]$ mysql -h database-1.c1kac4qykabw.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 28
Server version: 8.4.3 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

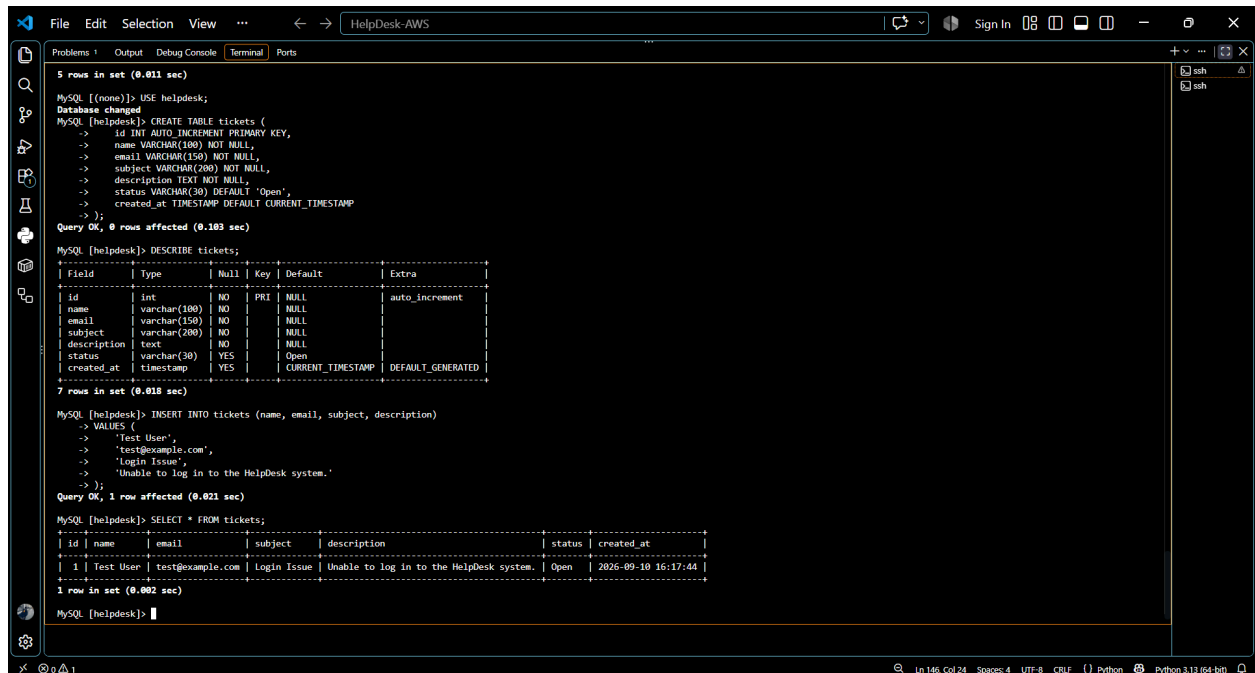
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.054 sec)

MySQL [(none)]> USE helpdesk;
ERROR 1049 (42000): Unknown database 'helpdesk'
MySQL [(none)]> CREATE DATABASE helpdesk;
Query OK, 1 row affected (0.049 sec)

MySQL [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| helpdesk |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.011 sec)

MySQL [(none)]> USE helpdesk;
Database changed
MySQL [helpdesk]>
```

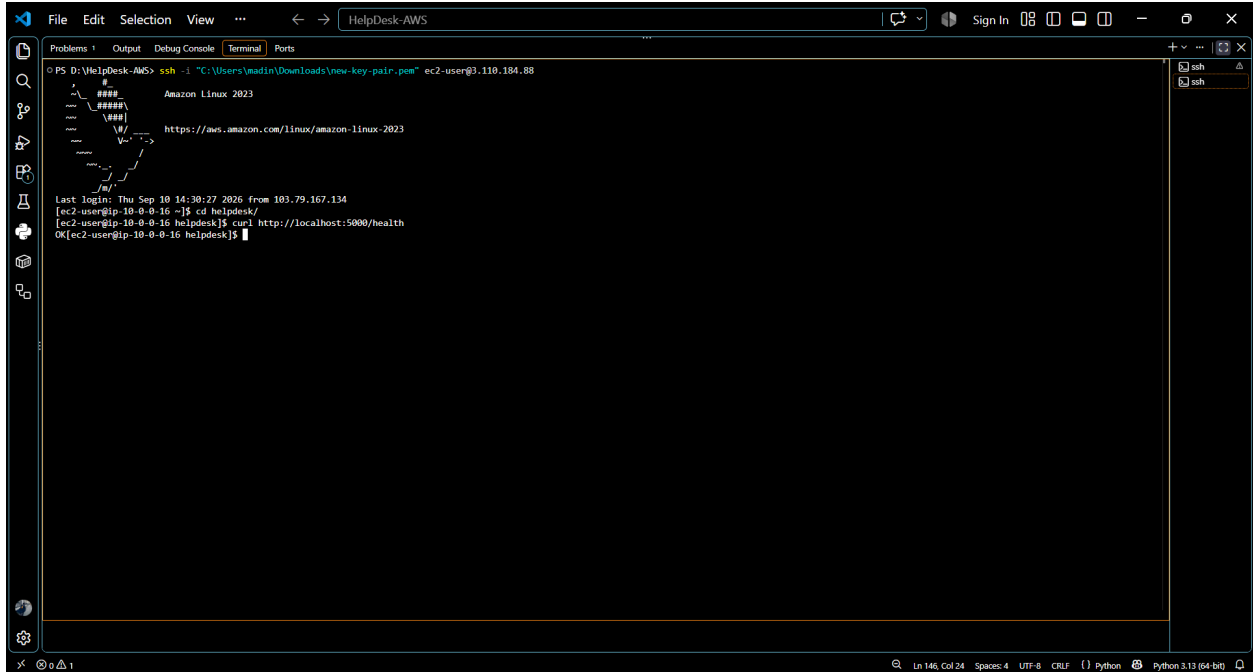


```
File Edit Selection View ... HelpDesk-AWS
Problems Output Debug Console Terminal Ports
5 rows in set (0.011 sec)
MySQL [(none)]> USE helpdesk;
Database changed
MySQL [helpdesk]> CREATE TABLE tickets (
-> id INT AUTO INCREMENT PRIMARY KEY,
-> name VARCHAR(100) NOT NULL,
-> email VARCHAR(150) NOT NULL,
-> subject VARCHAR(200) NOT NULL,
-> description TEXT NOT NULL,
-> status VARCHAR(50) DEFAULT 'Open',
-> created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
-> );
Query OK, 0 rows affected (0.103 sec)
MySQL [helpdesk]> DESCRIBE tickets;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| id | int | NO | PRI | NULL | auto_increment |
| name | varchar(100) | NO | | NULL | |
| email | varchar(150) | NO | | NULL | |
| subject | varchar(200) | NO | | NULL | |
| description | text | NO | | NULL | |
| status | varchar(50) | YES | | Open | |
| created_at | timestamp | YES | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+
7 rows in set (0.018 sec)
MySQL [helpdesk]> INSERT INTO tickets (name, email, subject, description)
-> VALUES (
-> 'Test User',
-> 'test@example.com',
-> 'Login Issue',
-> 'Unable to log in to the HelpDesk system.'
-> );
Query OK, 1 row affected (0.021 sec)
MySQL [helpdesk]> SELECT * FROM tickets;
+-----+-----+-----+-----+-----+-----+
| id | name | email | subject | description | status | created_at |
+-----+-----+-----+-----+-----+-----+
| 1 | Test User | test@example.com | Login Issue | Unable to log in to the HelpDesk system. | Open | 2026-09-10 16:17:44 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.002 sec)
MySQL [helpdesk]> |
```

Load Balancer Test

The Application Load Balancer was tested as the public entry point for the application.

The ALB forwarded requests to the target group on port 5000.



```
PS D:\HelpDesk-AWS> ssh -i "C:\Users\madin\Downloads\new-key-pair.pem" ec2-user@3.110.184.88
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Thu Sep 10 14:30:27 2026 from 103.79.167.134
[ec2-user@ip-10-0-0-16 ~]$ cd helpdesk/
[ec2-user@ip-10-0-0-16 helpdesk]$ curl http://localhost:5000/health
OK[ec2-user@ip-10-0-0-16 helpdesk]$
```

Target Health Test

The target group was monitored to determine whether the EC2 instances were healthy.

This testing revealed that application-level health depends on the Flask service actually running on the instance.

22. Problems Encountered and Troubleshooting

The project included several real-world troubleshooting situations.

RDS Subnet Capacity Problem

During the initial RDS creation attempt, AWS returned an error indicating that there was insufficient capacity for the selected configuration and subnet availability.

The issue was addressed by creating and configuring an appropriate RDS DB subnet group with subnets across Availability Zones.

This demonstrated an important AWS networking concept: managed services such as RDS have their own subnet requirements and do not simply use any arbitrary subnet.

Application Connection Problem

At one stage, attempting to access:

`127.0.0.1:5000`

from the local computer resulted in a connection error.

The reason was that `127.0.0.1` refers to the local computer itself, not the remote EC2 instance.

The application was instead accessed through the EC2 public IP and later through the Application Load Balancer.

This demonstrated the difference between:

localhost

and

a remote EC2 server address.

ALB Health Check Problem

The target group later showed unhealthy targets.

Investigation of one of the newly launched Auto Scaling instances showed that no Python process was running.

The following command was used:

```
ps aux | grep python
```

The result showed that Flask was not running.

The local health check also failed:

```
curl http://localhost:5000/health
```

The result was:

```
curl: (7) Failed to connect to localhost port 5000
```

The Cloud-init log was then inspected:

```
sudo cat /var/log/cloud-init-output.log
```

The log showed that the expected application installation/startup commands had not executed.

Therefore, the issue was identified as a Launch Template/User Data configuration problem rather than an ALB networking problem.

This was an important troubleshooting result because it demonstrated that an EC2 instance can successfully launch while the application inside the instance fails to start.

The project was intentionally stopped at this stage rather than hiding the issue or claiming that the Auto Scaling deployment was fully operational.

23. Security Considerations

Security was considered throughout the project.

Security Groups

Security Groups were used to control network access to EC2 resources.

Only required application traffic was allowed.

IAM

IAM roles were used instead of embedding long-term AWS credentials directly into application servers wherever applicable.

RDS

The database was deployed using Amazon RDS instead of exposing a database process directly from an EC2 server.

Network Isolation

The application infrastructure was placed inside a dedicated VPC.

SSH Access

SSH access was used for administration and required a configured key pair.

In a production environment, SSH access should be further restricted using a bastion host, Systems Manager Session Manager or other controlled administrative access mechanisms.

Production Improvements

A production deployment would require additional security measures, including:

- HTTPS/TLS using an ACM certificate
- More restrictive Security Group rules
- Private application subnets
- Private RDS subnets
- Secrets Manager for database credentials
- IAM least-privilege policies
- WAF protection
- Centralised logging
- Automated patching
- Proper CI/CD
- Infrastructure as Code

These features were outside the scope of this beginner/intermediate portfolio project.

24. Cost Management and Resource Cleanup

Cost management was an important part of the project because AWS resources can continue generating charges after testing has finished.

After completing the practical testing, active resources were cleaned up.

The following resources were terminated or deleted as appropriate:

- Auto Scaling Group
- EC2 instances
- Application Load Balancer
- Target Group
- Launch Template
- RDS database
- S3 resources that were no longer required
- CloudWatch resources that were created specifically for the project
- Project-specific networking resources where no longer required

The cleanup process demonstrated an important cloud engineering practice:

Infrastructure should not be left running unnecessarily after testing is complete.

For learning environments, checking the AWS console after every project is especially important to prevent unexpected costs.

[SCREENSHOT 34 — Final AWS resource cleanup / empty resource state]

25. Skills Demonstrated

This project demonstrated practical knowledge in several areas.

AWS

- Amazon EC2
- Amazon VPC
- Subnets

- Internet Gateway
- Route Tables
- Security Groups
- Amazon RDS
- Amazon S3
- Application Load Balancer
- Target Groups
- Auto Scaling
- Launch Templates
- CloudWatch
- IAM

Linux

- SSH
- Linux command-line administration
- Package installation
- Process monitoring
- Service troubleshooting
- File editing using Vim
- Network testing
- Application troubleshooting

Networking

- IPv4 CIDR ranges
- Public subnets
- Availability Zones
- Routing
- Internet Gateway
- Ports
- Security Groups
- Load balancing
- Application health checks

Python

- Python application execution
- Flask
- MySQL connector
- API/health endpoint
- Application configuration

Database

- MySQL
- Database creation
- Table creation
- SQL queries
- Data insertion
- Database verification

Cloud Operations

- Infrastructure deployment
 - Health monitoring
 - Troubleshooting
 - Resource cleanup
 - Cost awareness
 - High-availability concepts
 - Auto Scaling concepts
-

26. Project Outcome

The HelpDesk project successfully demonstrated the deployment of a cloud-based Flask application using multiple AWS services.

The application was successfully deployed to EC2 and connected to an Amazon RDS MySQL database.

Database creation, table creation, ticket insertion and ticket retrieval were successfully tested.

The Application Load Balancer and Target Group were configured to provide application traffic distribution and health checking.

A two-instance Auto Scaling architecture was also configured.

During the final Auto Scaling test, the automatically launched instances failed their application health checks because the Flask application was not started by the Launch Template's User Data configuration.

This issue was investigated using Linux process inspection, local HTTP testing and Cloud-init logs.

Although the Auto Scaling portion was not fully operational at the end of the project, the troubleshooting process successfully identified the root cause as an application bootstrap/User Data issue.

The project therefore provided practical experience not only in creating AWS infrastructure but also in diagnosing failures between infrastructure and application layers.

27. Conclusion

The HelpDesk AWS Cloud Deployment Project provided practical hands-on experience with designing, deploying, testing and troubleshooting a cloud-based web application.

The project combined a Flask application with Amazon EC2, Amazon RDS, Amazon S3, Application Load Balancing, Auto Scaling, VPC networking, IAM and CloudWatch.

One of the most important lessons from the project was that successful infrastructure deployment does not automatically mean that an application is operational.

An EC2 instance can be running while the application inside it is stopped. Similarly, an Application Load Balancer can be configured correctly while its targets remain unhealthy because the application is not listening on the expected port or responding to the health-check endpoint.

The project therefore provided experience across both infrastructure and application layers.

The database component demonstrated how a managed AWS database can be separated from application compute. The load balancer demonstrated how traffic can be distributed between multiple instances. The Auto Scaling configuration demonstrated how AWS can automatically manage application capacity. CloudWatch demonstrated the importance of monitoring cloud resources.

The final troubleshooting exercise involving the Launch Template and User Data was particularly valuable because it exposed a realistic operational problem rather than a perfectly controlled tutorial environment.

Overall, the HelpDesk project strengthened practical skills in AWS cloud infrastructure, Linux administration, networking, Python/Flask deployment, database management, load balancing, monitoring, troubleshooting and cloud cost management.

The project also established a foundation for future improvements such as Terraform infrastructure as code, CI/CD automation, HTTPS, private subnets, AWS Secrets Manager, improved Auto Scaling configuration and more advanced monitoring.

Project Status: Completed as a Cloud Engineering Portfolio Project

AWS Resources: Cleaned Up

Region: ap-south-1 (Mumbai)
